

1. Binomial coefficients are a family of positive integers that have a number of useful properties and they can be defined in several ways. One way to define them is as an indexed recursive function, $C(n, k)$, where the “C” stands for “choice” or “combinations.” In this case, the definition is as follows:

$$C(n, 0) = 1$$

$$C(n, n) = 1$$

$$C(n, k) = C(n-1, k-1) + C(n-1, k) \quad 0 < k < n$$

Describe a scheme for computing $C(n, k)$ using memoization.

Solution: For a 2D table, according to the recurrence, each time we look at the element at “upper left” and “upper” with respect to current element, and as we have $k < n$, we only need to fill the “bottom left” half of the memory table.

BinomialCoefficient(n, k):

Initialize a 2D array dp of size $(n+1) \times (k)$

for $i \leftarrow 0$ to n

$dp[i][0] \leftarrow 1$ *⟨⟨ $C(n, 0) = 1$ for all n ⟩⟩*

$dp[i][i] \leftarrow 1$ *⟨⟨ $C(n, n) = 1$ for all n ⟩⟩*

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to $i-1$

$dp[i][j] \leftarrow dp[i-1][j-1] + dp[i-1][j]$

return $dp[n][k]$

Runtime: $O(nk)$



2. Suppose we are given a collection $A = \{a_1, a_2, \dots, a_n\}$ of n positive integers that add up to N . Design an $O(nN)$ -time algorithm for determining whether there is a subset $B \subseteq A$ such that $\sum_{a_i \in B} a_i = \sum_{a_i \in A-B} a_i$.

Solution: Does there exist a subset that sums to $\frac{N}{2}$

English Description $E(i, j)$:

Let $E(i, j)$ decide whether there exists a subset of $A[i \dots n]$ that sums to j .

Recursive Relation:

$$E(i, j) = E(i + 1, j) \vee E(i + 1, j - A[i])$$

- **General Case:** The relation holds for all valid i and j .
- **Base Case:** $E(i, 0) = \text{True}$ for $i \in 1 \dots n + 1$.
- **Invalid Case:** $E(i, j) = \text{False}$ if $j < 0$.

Answer:

The final answer will be $E(1, \frac{N}{2})$.

Memoization:

- Use a 2D array to store results: decreasing i , increasing j .
- **Space Complexity:** $O(nN)$.

Time Complexity:

- $O(n)$ options for i , $O(N)$ options for j , and each takes $O(1)$ comparisons.
- **Time Complexity:** $O(nN)$.



3. Matrix multiplication is associative, and so all parenthesizations yield the same product. A product of matrices is *fully parenthesized* if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses. For example, if the chain of matrices is $\langle A_1, \dots, A_4 \rangle$, then we can fully parenthesize the product in five distinct ways:

- $(A_1, (A_2, (A_3, A_4)))$
- $(A_1, ((A_2, A_3), A_4))$
- $((A_1, A_2), (A_3, A_4))$
- $((A_1, (A_2, A_3)), A_4)$
- $(((A_1, A_2), A_3), A_4)$

How we parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product. For example, if we have three matrices with dimensions: A_1 $[10 \times 100]$, A_2 $[100 \times 5]$, A_3 $[5 \times 50]$, then $((A_1 A_2) A_3)$ takes 7,500 scalar multiplications while $(A_1 (A_2 A_3))$ takes 75,000 scalar multiplications.

Given a chain of n matrices $A_1 A_2 \dots A_n$, where matrix A_i has dimension $r_i \times c_i$, provide a dynamic programming algorithm that finds the minimum number of scalar multiplications required to evaluate the product of the matrices.

Solution: (a) For any two consecutive matrices M_i and M_{i+1} , $c_i = r_{i+1}$.

(b) Multiplying two matrices $M_i M_j$ requires $r_i c_i c_j (= r_i r_j c_j)$ scalar multiplications.

(c) Multiplying matrices $M_i, M_{i+1}, \dots, M_j$ results in a matrix with dimension $r_i \times c_j$.

Consider a matrix chain $M_1 M_2 \dots M_n$. There are $n - 1$ matrix multiplications to compute, and we want to order the $n - 1$ matrix multiplications in a way that minimizes the number of scalar multiplications. Suppose we compute the k -th multiplication (the multiplication between M_k and M_{k+1}) at the very last. Then, the cost (the number of scalar multiplications) of the k -th multiplication would be $r_1 c_k c_n$, since the matrices $M_1 \dots M_k$ and $M_{k+1} \dots M_n$ would be multiplied separately before the k -th multiplication, giving the dimensions $r_1 \times c_k$ and $r_{k+1} \times c_n$ respectively. Therefore, the minimum cost we can get while computing the k -th multiplication at last would be $MSM(1, k) + MSM(k + 1, n) + r_1 c_k c_n$, where $MSM(i, j)$ is the minimum total cost of the multiplications on $M_i \dots M_j$. Based on the above observation, we can construct the following recurrence.

$$MSM(i, j) = \begin{cases} 0 & \text{if } i \geq j \\ \min_{i \leq k < j} MSM(i, k) + MSM(k + 1, j) + r_i c_k c_j & \text{otherwise} \end{cases}$$

As mentioned, $MSM(i, j)$ represents the minimum number of scalar multiplications required to compute $M_i M_{i+1} \dots M_j$, and therefore $MSM(1, n)$ would be the final answer we are looking for.

To get a dynamic programming algorithm, we use a 2D array $MSM[1..n, 1..n]$ for memoization, filling out the entries in decreasing i and increasing j order.

```
MSM( $A[1..n]$ ):  
  for  $i \leftarrow 1$  to  $n$                                 «Base cases»  
     $MSM[i, j] \leftarrow 0$   
  for  $i \leftarrow n$  down to 1  
    for  $j \leftarrow i + 1$  to  $n$   
       $min \leftarrow \infty$   
      for  $k \leftarrow i$  to  $j - 1$   
         $c \leftarrow MSM[i, k] + MSM[k + 1, j] + r_i c_k c_j$   
        if  $c < min$   
           $min \leftarrow c$   
       $MSM[i, j] \leftarrow min$   
  return  $MSM[1, n]$ 
```

Since the amount of work at each iteration is constant, the runtime of the algorithm is $O(n^3)$. ■